Московский Авиационный Институт

(Национальный Исследовательский Университет)

Институт №8 “Компьютерные науки и прикладная математика”

Кафедра №806 “Вычислительная математика и программирование”

Лабораторная работа №1 по курсу

«Операционные системы»

Группа: М8О-212БВ-25

Студент: Сергина А.А.

Преподаватель: Бахарев В.Д.

Оценка: _____

Дата: 04.10.26

Москва, 2026

Постановка задачи

Вариант 18.

Родительский процесс создает два дочерних процесса. Первой строкой пользователь в консоль родительского процесса вводит имя файла, которое будет использовано для открытия File с таким именем на запись для child1. Аналогично для второй строки и процесса child2. Родительский и дочерний процесс должны быть представлены разными программами.

Родительский процесс принимает от пользователя строки произвольной длины и пересылает их в pipe1 или в pipe2 в зависимости от правила фильтрации. Процесс child1 и child2 производят работу над строками. Процессы пишут результаты своей работы в стандартный вывод.

Правило фильтрации: нечетные строки отправляются в pipe1, четные в pipe2. Дочерние процессы удаляют все гласные из строк.

Общий метод и алгоритм решения

Использованные системные вызовы:

pid_t fork(void); – создает дочерний процесс. Используется в родительском процессе для создания двух потомков (child1 и child2), которые будут выполнять фильтрацию строк.

● int pipe(int *fd); – создает неименованный канал (pipe) для однонаправленной передачи данных. Возвращает два файловых дескриптора: fd[0] для чтения и fd[1] для записи. В программе создаются два канала: pipe1 и pipe2, по которым родитель передает строки дочерним процессам.

● int dup2(int oldfd, int newfd); – дублирует файловый дескриптор oldfd в newfd. Если newfd уже открыт, он предварительно закрывается. Используется в дочерних процессах для перенаправления стандартного ввода (STDIN_FILENO) на чтение из pipe, а также для перенаправления стандартного вывода (STDOUT_FILENO) в открытый файл.

● int open(const char *pathname, int flags, mode_t mode); – открывает или создает файл. В дочерних процессах используется для открытия файла (имя которого передано аргументом) на запись с флагами O_WRONLY | O_CREAT | O_TRUNC и правами 0600.

● ssize_t read(int fd, void *buf, size_t count); – читает данные из файлового дескриптора. Родитель читает строки из стандартного ввода (STDIN_FILENO), дочерние процессы читают данные из pipe (через перенаправленный STDIN_FILENO).

● ssize_t write(int fd, const void *buf, size_t count); – записывает данные в файловый дескриптор. Родитель пишет строки в pipe1 или pipe2. Дочерние процессы пишут результат в стандартный вывод (перенаправленный в файл).

● int close(int fd); – закрывает файловый дескриптор. Используется для закрытия ненужных концов pipe в родительском и дочерних процессах, а также для закрытия файла после dup2.

● int execlp(const char *file, const char *arg, ...); – заменяет текущий образ процесса новым. В дочерних процессах вызывается для запуска исполняемых файлов ./child1 и ./child2 с передачей имени выходного файла в качестве аргумента.

● pid_t waitpid(pid_t pid, int *wstatus, int options); – ожидает завершения дочернего процесса. Родитель вызывает waitpid для обоих потомков, чтобы дождаться их завершения перед выходом.

● void exit(int status); – завершает процесс с указанным кодом возврата. Используется для аварийного завершения при ошибках.

Родительский процесс сначала читает две строки из стандартного ввода — это имена файлов для child1 и child2. Создает два канала pipe1 и pipe2. Затем с помощью fork создает два дочерних процесса. В child1 перенаправляет стандартный ввод на чтение из pipe1, закрывает лишние дескрипторы и запускает программу child1, передавая ей имя первого файла. Аналогично для child2, но с pipe2 и вторым файлом. Родитель закрывает концы каналов, предназначенные для чтения, и начинает построчно читать входные данные из стандартного ввода. Каждая строка анализируется: если её номер нечетный, она записывается в pipe1, если четный — в pipe2. В конце родитель закрывает записывающие концы каналов и ожидает завершения дочерних процессов. Дочерние процессы открывают указанный файл на запись, перенаправляют свой стандартный вывод в этот файл. Затем читают данные из pipe, удаляют все гласные буквы (только латинские!) и записывают результат в файл. Работают до тех пор, пока из канала не поступит сигнал EOF.

Код программы

parent.c

```
#include <fcntl.h>
#include <stdlib.h>
#include <sys/wait.h>
#include <unistd.h>
int main(int argc, char *argv[]) {
    char out1[256];
    char out2[256];
    int n1 = read(0, out1, 255);
    int n2 = read(0, out2, 255);
    if (n1 <= 0 || n2 <= 0) {
        write(2, "Ошибка ввода\n", 13);
        exit(-1);
    }
    out1[n1 - 1] = '\0';
    out2[n2 - 1] = '\0';
    int pipe1[2];
```

```
int pipe2[2];
if (pipe(pipe1) == -1 || pipe(pipe2) == -1) {
    write(2, "Ошибка создания pipe\n", 20);
    exit(-1);
}
const pid_t child1 = fork();
if (child1 == -1) {
    write(2, "Ошибка fork\n", 11);
    exit(-1);
} else if (child1 == 0) {
    close(pipe1[1]);
    close(pipe2[0]);
    close(pipe2[1]);
    dup2(pipe1[0], STDIN_FILENO);
    close(pipe1[0]);
    execlp("./child1", "child1", out1, NULL);
    write(2, "Ошибка запуска child1\n", 22);
    exit(-1);
}
const pid_t child2 = fork();
if (child2 == -1) {
```

```
    write(2, "Ошибка fork\n", 11);
    exit(-1);
} else if (child2== 0) {
    close(pipe2[1]);
    close(pipe1[0]);
    close(pipe1[1]);
    dup2(pipe2[0], STDIN_FILENO);
    close(pipe2[0]);
    execlp("./child2", "child2", out2, NULL);
    write(2, "Ошибка запуска child2\n", 22);
    exit(-1);
}
close(pipe1[0]);
close(pipe2[0]);
char buffer[2048];
char line[1024];
int ind = 0;
int lineNum = 1;
int bytes;
while (1) {
    bytes = read(STDIN_FILENO, buffer, sizeof(buffer));
```

```
if (bytes == -1) {
    write(2, "Ошибка чтения\n", 15);
    exit(-1);
}
if (bytes == 0) {
    break;
}
for (int i = 0; i < bytes; ++i) {
    char c = buffer[i];
    if (c == '\n') {
        line[ind] = '\0';
        if (lineNum % 2 != 0) {
            write(pipe1[1], line, ind);
            write(pipe1[1], "\n", 1);
        } else {
            write(pipe2[1], line, ind);
            write(pipe2[1], "\n", 1);
        }
        ++lineNum;
        ind = 0;
    } else {
```

```
            if (ind < sizeof(line) - 1) {
                line[ind++] = c;
            }
        }
    }
}
if (ind > 0) {
    line[ind] = '\0';
    if (lineNum % 2 != 0) {
        write(pipe1[1], line, ind);
        write(pipe1[1], "\n", 1);
    } else {
        write(pipe2[1], line, ind);
        write(pipe2[1], "\n", 1);
    }
}
close(pipe1[1]);
close(pipe2[1]);
waitpid(child1, NULL, 0);
waitpid(child2, NULL, 0);
return 0;
```

```
}
```

child1.c

```
#include <fcntl.h>
#include <stdlib.h>
#include <sys/wait.h>
#include <unistd.h>
int IsG(char c) {
    if (c >= 'A' && c <= 'Z') {
        c += 32;
    }
    if (c == 'a' || c == 'e' || c == 'i' || c == 'o' || c == 'u' || c == 'y' ||
        c == 'A' || c == 'E' || c == 'I' || c == 'O' || c == 'U' || c == 'Y') {
        return 1;
    }
    return 0;
}

int Result(char* s, char* res, int len) {
    int k = 0;
    for (int i = 0; i < len; ++i) {
        if (!IsG(s[i])) {
```

```
            res[k] = s[i];
            ++k;
        }
    }
    res[k] = '\0';
    return k;
}

int main(int argc, char* argv[]) {
    if (argc != 2) {
        write(2, "Использование: ./pchild1 <входной_файл>\n", 36);
        exit(-1);
    }
    int out = open(argv[1], O_WRONLY | O_CREAT | O_TRUNC, 0600);
    if (out == -1) {
        write(2, "Ошибка открытия выходного файла\n", 33);
        exit(-1);
    }
    if (dup2(out, STDOUT_FILENO) == -1) {
        write(2, "Ошибка dup2\n", 12);
        exit(-1);
```

```
    }
    close(out);
    char buffer[1024];
    char result[1024];
    int bytes;
    while (1) {
        bytes = read(STDIN_FILENO, buffer, sizeof(buffer));
        if (bytes == -1) {
            write(2, "Ошибка чтения\n", 15);
            exit(-1);
        } else if (bytes == 0) {
            break;
        }
        int len = Result(buffer, result, bytes);
        if (write(STDOUT_FILENO, result, len) == -1) {
            write(2, "Ошибка записи\n", 15);
            exit(-1);
        }
    }
    return 0;
}
```

child2.c

```
#include <fcntl.h>
#include <stdlib.h>
#include <sys/wait.h>
#include <unistd.h>

int IsG(char c) {
    if (c >= 'A' && c <= 'Z') {
        c += 32;
    }
    if (c == 'a' || c == 'e' || c == 'i' || c == 'o' || c == 'u' || c == 'y' ||
        c == 'A' || c == 'E' || c == 'I' || c == 'O' || c == 'U' || c == 'Y') {
        return 1;
    }
    return 0;
}

int Result(char* s, char* res, int len) {
    int k = 0;
    for (int i = 0; i < len; ++i) {
        if (!IsG(s[i])) {
```

```
            res[k] = s[i];
            ++k;
        }
    }
    res[k] = '\0';
    return k;
}

int main(int argc, char* argv[]) {
    if (argc != 2) {
        write(2, "Использование: ./pchild2 <входной_файл>\n", 36);
        exit(-1);
    }
    int out = open(argv[1], O_WRONLY | O_CREAT | O_TRUNC, 0600);
    if (out == -1) {
        write(2, "Ошибка открытия выходного файла\n", 33);
        exit(-1);
    }
    if (dup2(out, STDOUT_FILENO) == -1) {
        write(2, "Ошибка dup2\n", 12);
        exit(-1);
```

```
}
close(out);


char buffer[1024];
char result[1024];
int bytes;
while (1) {
    bytes = read(STDIN_FILENO, buffer, sizeof(buffer));
    if (bytes == -1) {
        write(2, "Ошибка чтения\n", 15);
        exit(-1);
    } else if (bytes == 0) {
        break;
    }
    int len = Result(buffer, result, bytes);
    if (write(STDOUT_FILENO, result, len) == -1) {
        write(2, "Ошибка записи\n", 15);
        exit(-1);
    }
}
return 0;
```

Протокол работы программы

srg_anya@anya:~/projects/OS/lab1$ ./parent
test1.txt
test2.txt
The parent process creates two child processes.
The user enters the name of a file in the console of the parent process on the first line.
This file will be used for opening a File with such a name for writing for child1.
Similarly, for the second line and the child2 process.
The parent and child processes must be represented by different programs.
The parent process receives strings of arbitrary length from the user.
It forwards them to pipe1 or pipe2 depending on the filtering rule.
The child1 and child2 processes perform work on the strings.
The processes write the results of their work to standard output.
Variant 18) Filtering rule: odd lines are sent to pipe1, even lines to pipe2.
Child processes remove all vowels from the strings.

srg_anya@anya:~/projects/OS/lab1$ cat test1.txt
Th prnt prcss crts tw chld prcsss.
Ths fl wll b sd fr pnng Fl wth sch nm fr wrtng fr chld1.
Th prnt nd chld prcsss mst b rprsntd b dffrnt prgrms.
t frwrds thm t pp1 r pp2 dpndng n th fltrng rl.
Th prcsss wrt th rslts f thr wrk t stndrd tpt.
Chld prcsss rmv ll vwls frm th strngs.

srg_anya@anya:~/projects/OS/lab1$ cat test2.txt
Th sr ntrs th nm f fl n th cnsl f th prnt prcss n th frst ln.

Smlrl, fr th scnd ln nd th chld2 prcss.

Th prnt prcss rcvs strngs f rbtrr lngth frm th sr.

Th chld1 nd chld2 prcsss prfrm wrk n th strngs.

Vrnt 18) Fltrng rl: dd lns r snt t pp1, vn lns t pp2.

Вывод

В ходе работы были изучены механизмы межпроцессного взаимодействия через неименованные каналы и создание дочерних процессов с помощью fork. Программа реализует удаление гласных из строк. Проблема возникла с выводом итогового результата, а именно, что нельзя построчно выводить результат из двух откртых файлов сразу (сначала выводится содержимое одного файла, потом другого).